

BABU BANARASI DAS UNIVERSITY



2026 - 2027

SCHOOL OF COMPUTER APPLICATIONS

NoSQL Project On

College Student Record Management System Using MongoDB

Submitted to:

Mr. Harendra Singh

Submitted by:

Joanna

Department: BCA (DS & AI)

Roll No: 67

Date: 25th September

Section: BCADS23

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled “College Student Record Management System Using MongoDB”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update and delete data. I also learned how different comparison and logical query operators such as equal to, greater than, less than, AND, OR and NOR are used inside MongoDB queries, along with methods like skip(), limit() and count().

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents instead of fixed rows and columns like traditional relational databases. This flexibility makes it easy to store real-world data such as student records, where each record can hold nested information.

In this project, a College Student Record Management System has been created using MongoDB. A database named College2 was created, and inside it a collection named Students2 was used to store the records of 52 students. Each student document contains the student's name, age, course, address (city and state), and marks.

Various MongoDB operations have been performed on this data, such as inserting multiple records at once, displaying records, updating a record, deleting a record, and searching records using comparison operators (\$eq, \$gt, \$lt, \$gte, \$lte, \$ne, \$in, \$nin) and logical operators (\$and, \$or, \$nor). The find() method was also used along with skip(), limit() and count() to control and summarise the output.

OBJECTIVES

- To understand the concept of NoSQL databases.
- To learn the basic features of MongoDB.
- To create a database and a collection.
- To insert multiple student records using insertMany().
- To display and read records using find() and pretty().
- To perform update and delete operations on documents.
- To use comparison operators such as \$eq, \$gt, \$lt, \$gte, \$lte and \$ne.
- To use the \$in and \$nin operators for matching multiple values.
- To understand logical operators — \$and, \$or and \$nor.
- To use skip(), limit() and count() to control and summarise query results.

1. CREATE / SWITCH DATABASE

MongoDB Query:

```
use("College2")
```

Explanation: This command creates a new database named College2 if it does not already exist, or switches to it if it does. All the collections and student records used in this project are stored inside this database, and the collection used to hold the records is named Students2.

2. INSERT STUDENT RECORDS

MongoDB Query:

```
db.Students2.insertMany([ { name: "Aarav Sharma", age: 20, course: "BCA",  
address: { city: "Lucknow", state: "Uttar Pradesh" }, marks: 82 }, { name:  
"Riya Verma", age: 19, course: "BCA", address: { city: "Kanpur", state:  
"Uttar Pradesh" }, marks: 76 }, ... and so on for 52 students ])
```

Explanation: The insertMany() method is used to insert multiple documents into the Students2 collection in a single command. Every document stores the details of one student — name, age, course, address (with city and state as a nested sub-document), and marks obtained. In this way, 52 student records were added to the collection in one go.

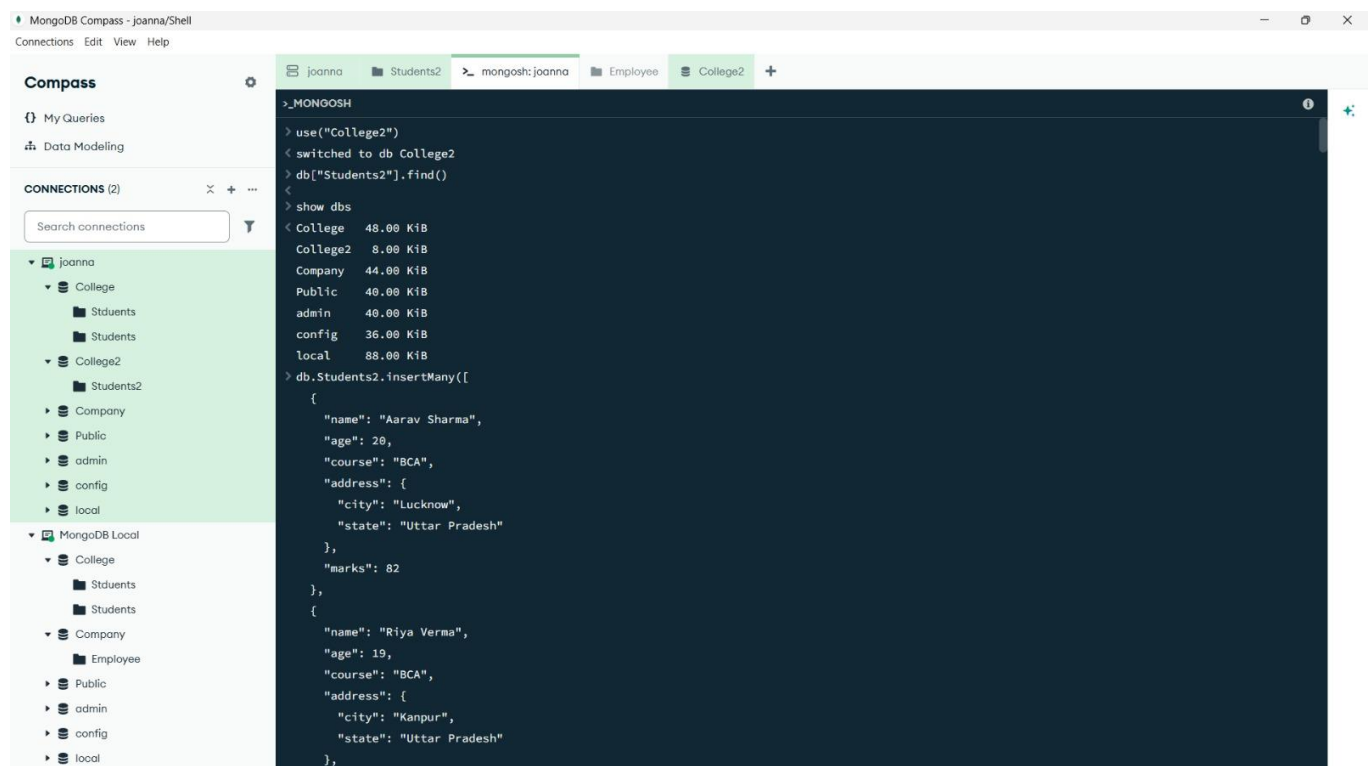


Fig 2.1 — Inserting student records using insertMany() in MongoDB Compass

3. DISPLAY ALL STUDENTS

MongoDB Query:

```
db.Students2.find()
```

Explanation: The `find()` method with no filter retrieves and displays every document currently stored in the `Students2` collection. This is the most basic and frequently used read operation in MongoDB.

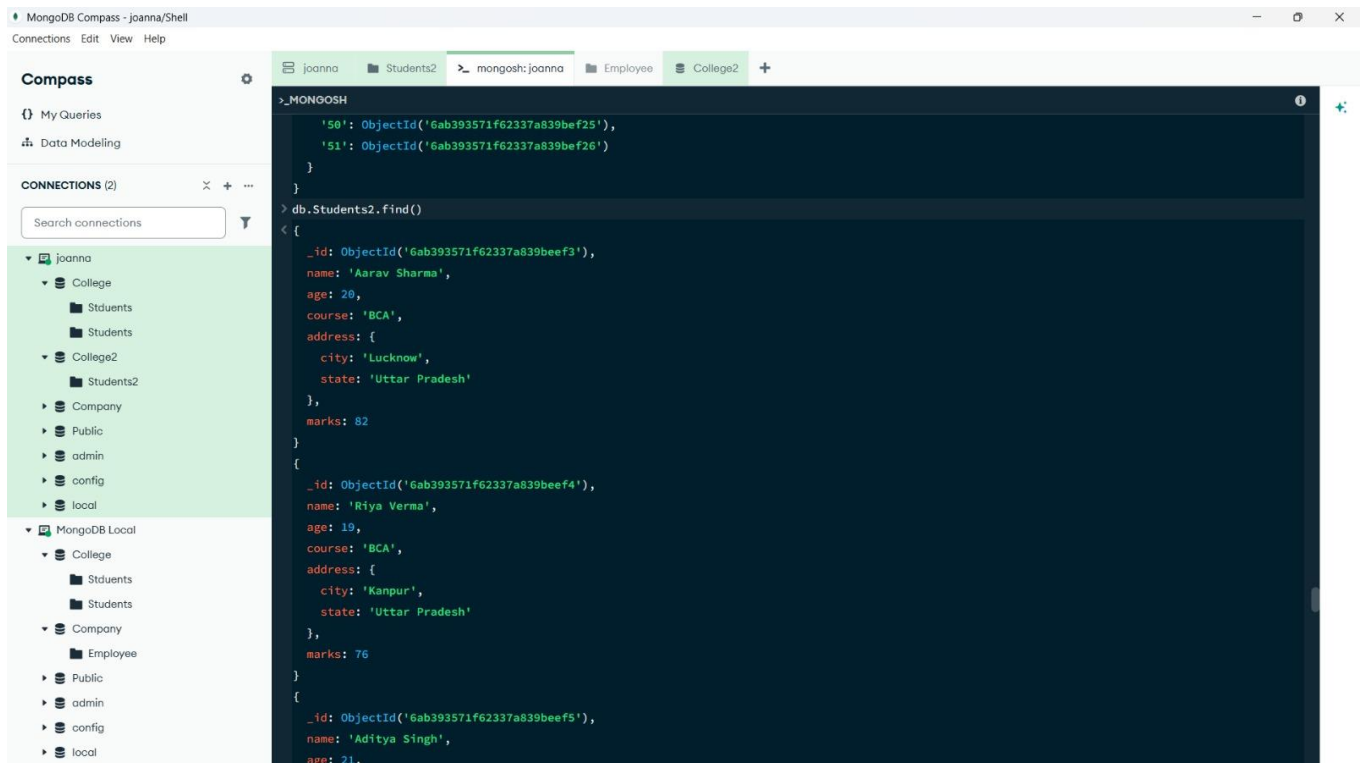


Fig 3.1 — Displaying all student documents using `find()`

4. DISPLAY RECORDS IN READABLE FORMAT

MongoDB Query:

```
db.Students2.find().pretty()
```

Explanation: The `pretty()` method formats the query output in a neat, indented structure instead of printing it as one continuous line, which makes nested fields such as address much easier to read.

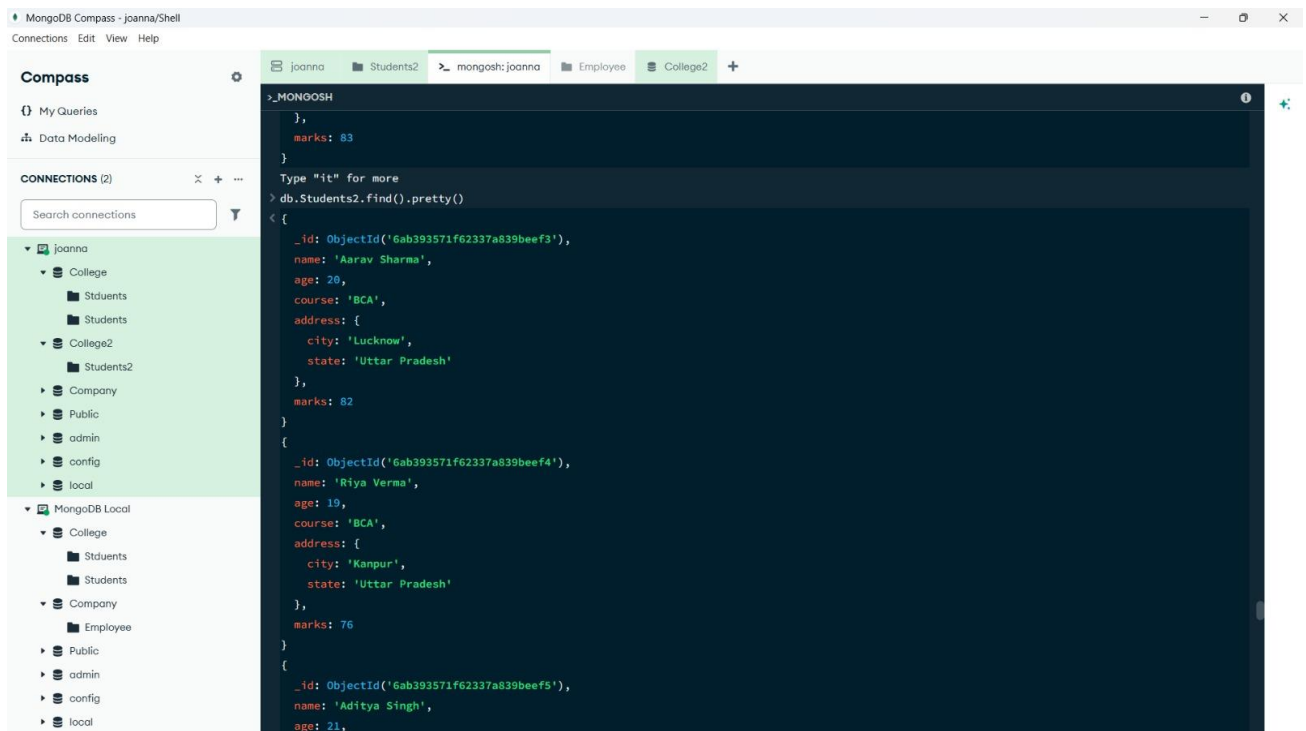


Fig 4.1 — Formatted output using `find().pretty()`

5. EQUAL TO — \$eq

MongoDB Query:

```
db.Students2.find({ age: { $eq: 19 } })
```

Explanation: The `$eq` operator matches documents where the value of a field is exactly equal to the specified value. Here, it returns all students whose age is exactly 19.

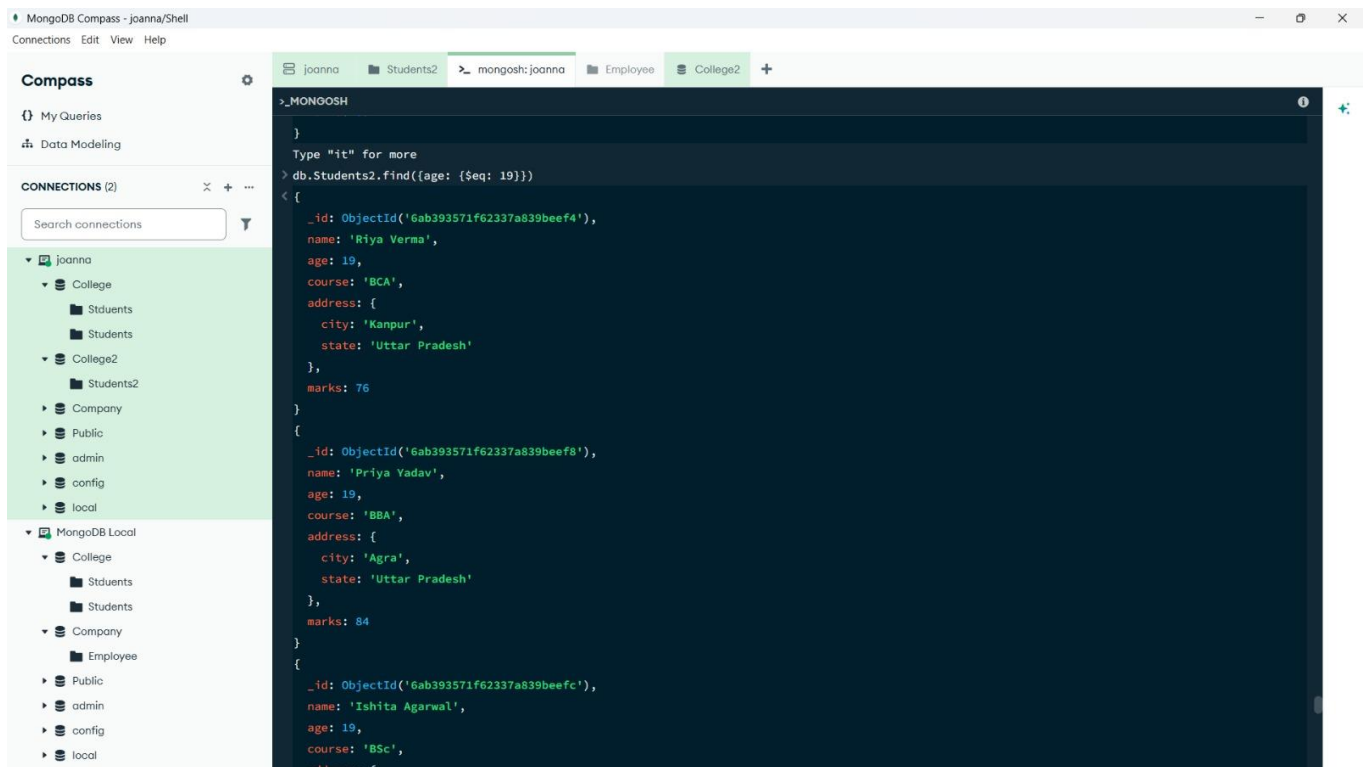


Fig 5.1 — Students with age equal to 19

6. GREATER THAN — \$gt

MongoDB Query:

```
db.Students2.find({ marks: { $gt: 80 } })
```

Explanation: The `$gt` operator returns documents where the given field has a value greater than the specified number. This query displays all students who scored more than 80 marks.

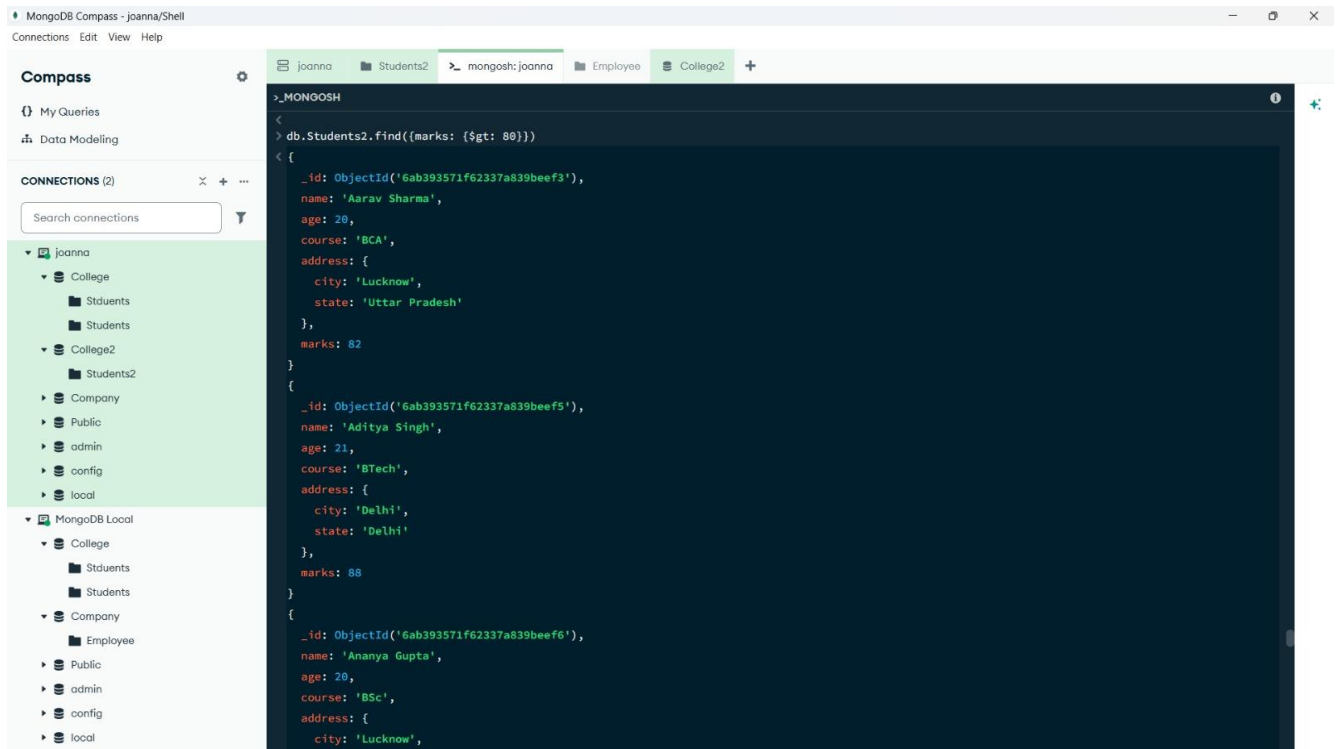


Fig 6.1 — Students with marks greater than 80

7. LESS THAN — \$lt

MongoDB Query:

```
db.Students2.find({ marks: { $lt: 80 } })
```

Explanation: The `$lt` operator returns documents where the field value is less than the specified number. This query lists all students who scored below 80 marks.

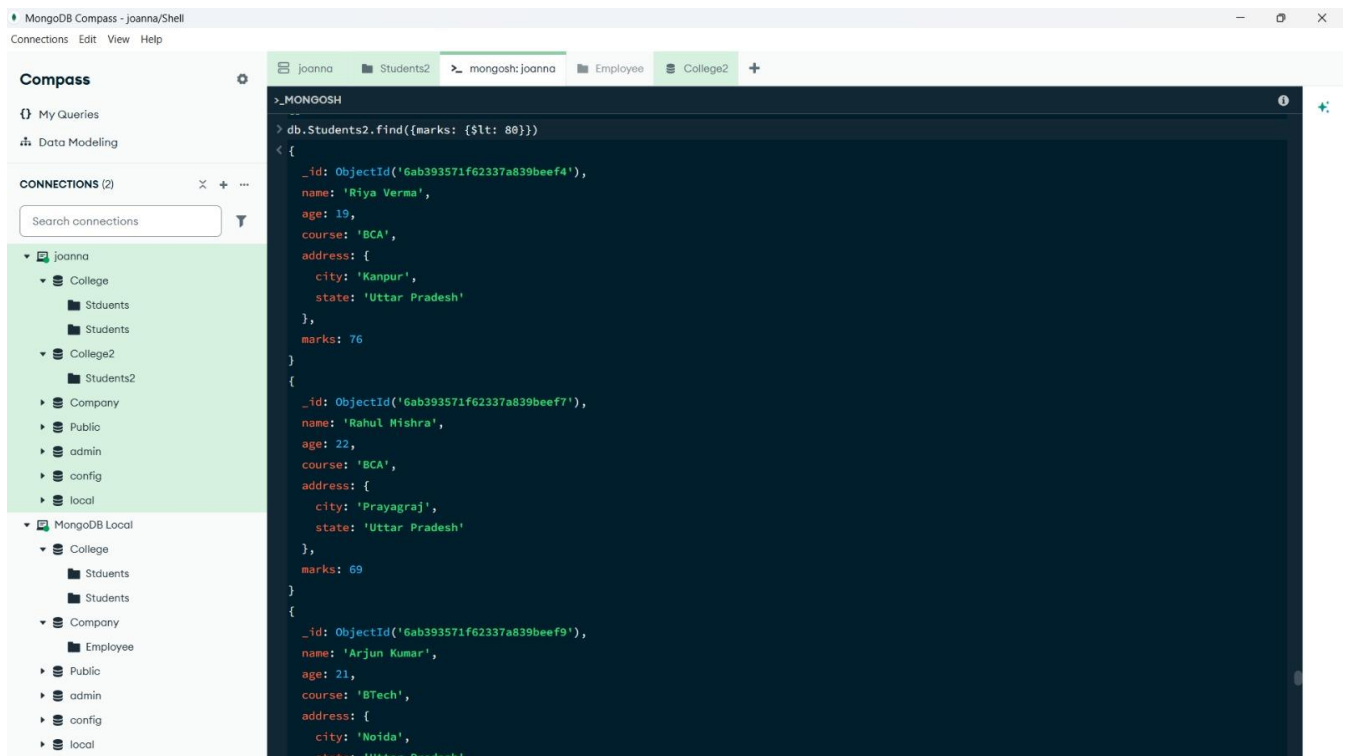


Fig 7.1 — Students with marks less than 80

8. GREATER THAN OR EQUAL TO — \$gte

MongoDB Query:

```
db.Students2.find({ age: { $gte: 22 } })
```

Explanation: The `$gte` operator matches documents where the field value is greater than or equal to the given value. This query finds all students aged 22 or above.

```
>_MONGOSH
},
  marks: 83
}
Type "it" for more
> db.Students2.find({age: {$gte: 22}})
< {
  _id: ObjectId('6ab393571f62337a839beef7'),
  name: 'Rahul Mishra',
  age: 22,
  course: 'BCA',
  address: {
    city: 'Prayagraj',
    state: 'Uttar Pradesh'
  },
  marks: 69
}
{
  _id: ObjectId('6ab393571f62337a839beefb'),
  name: 'Karan Mehta',
  age: 22,
  course: 'BCom',
  address: {
    city: 'Jaipur',
    state: 'Rajasthan'
  },
  marks: 71
}
{
  _id: ObjectId('6ab393571f62337a839beeff'),
  name: 'Vivek Joshi',
  age: 22,
```

Fig 8.1 — Students with age greater than or equal to 22

9. LESS THAN OR EQUAL TO — \$lte

MongoDB Query:

```
db.Students2.find({ marks: { $lte: 70 } })
```

Explanation: The \$lte operator matches documents where the field value is less than or equal to the given value. This query displays students who scored 70 marks or below.

```
>_MONGOSH
},
  marks: 71
}
> db.Students2.find({marks: {$lte: 70}})
< {
  _id: ObjectId('6ab393571f62337a839beef7'),
  name: 'Rahul Mishra',
  age: 22,
  course: 'BCA',
  address: {
    city: 'Prayagraj',
    state: 'Uttar Pradesh'
  },
  marks: 69
}
{
  _id: ObjectId('6ab393571f62337a839bef01'),
  name: 'Mohit Sharma',
  age: 21,
  course: 'BCA',
  address: {
    city: 'Meerut',
    state: 'Uttar Pradesh'
  },
  marks: 68
}
{
  _id: ObjectId('6ab393571f62337a839bef09'),
  name: 'Sahil Gupta',
  age: 22,
  course: 'BCA',
```

Fig 9.1 — Students with marks less than or equal to 70

10. NOT EQUAL TO — \$ne

MongoDB Query:

```
db.Students2.find({ city: { $ne: "Meerat" } })
```

Explanation: The \$ne operator returns documents where the field value is not equal to the specified value. This query is used to display all student records except the ones matching the given city value.

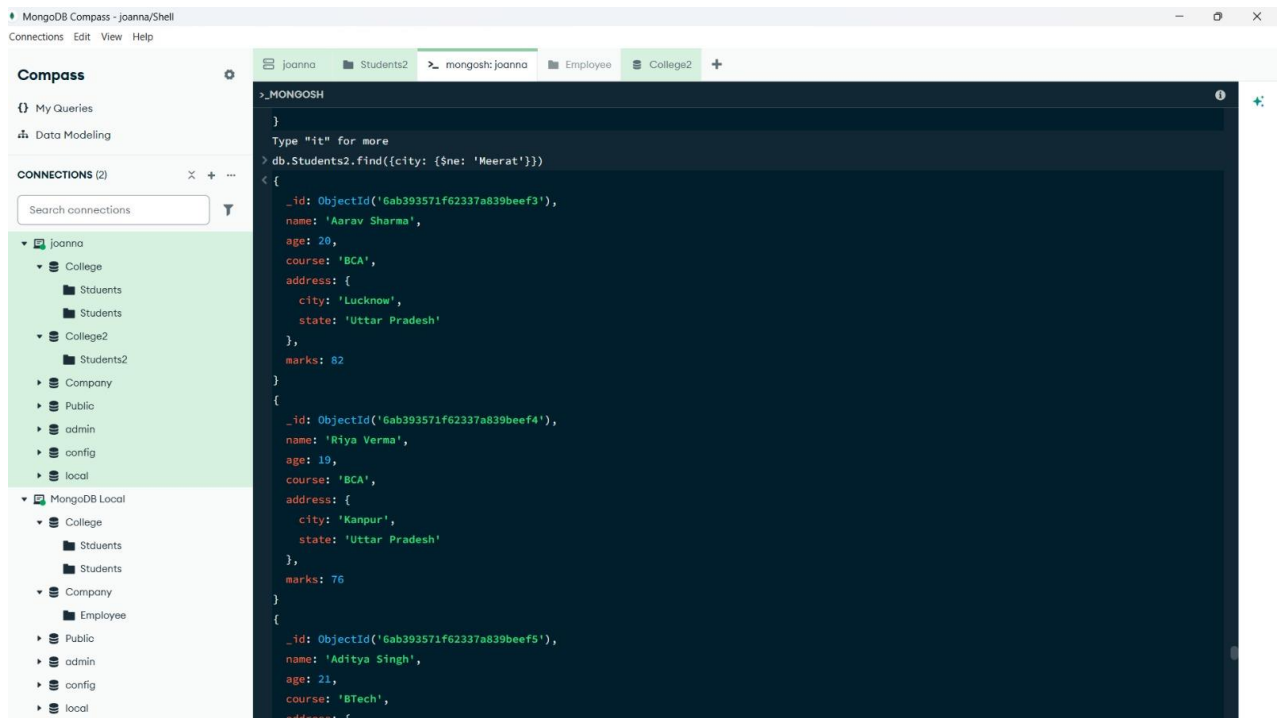


Fig 10.1 — Records where city is not equal to the given value

11. IN OPERATOR — \$in

MongoDB Query:

```
db.Students2.find({ "address.city": { $in: ["Chennai", "Agra"] } })
```

Explanation: The \$in operator matches documents where the field's value is equal to any value present inside the given array. This query returns all students who belong to either Chennai or Agra.

```
>_MONGOOSH
|
^
> db.Students2.find({"address.city": {$in: ["Chennai", "Agra"]}})
< {
  _id: ObjectId('6ab393571f62337a839bbee08'),
  name: 'Priya Yadav',
  age: 19,
  course: 'BBA',
  address: {
    city: 'Agra',
    state: 'Uttar Pradesh'
  },
  marks: 84
}
{
  _id: ObjectId('6ab393571f62337a839bbee09'),
  name: 'Meera Iyer',
  age: 21,
  course: 'BSc',
  address: {
    city: 'Chennai',
    state: 'Tamil Nadu'
  },
  marks: 94
}
{
  _id: ObjectId('6ab393571f62337a839bbee0a'),
  name: 'Harsh Vardhan',
  age: 21,
  course: 'BCA',
  address: {
    city: 'Agra',

```

Fig 11.1 — Students belonging to Chennai or Agra using \$in

12. NOT IN OPERATOR — \$nin

MongoDB Query:

```
db.Students2.find({ "address.city": { $nin: ["Lucknow"] } })
```

Explanation: The \$nin operator is the opposite of \$in — it matches documents where the field's value does not appear in the given array. This query displays all students who do not belong to Lucknow.

```
>_MONGOSH
> db.Students2.find({"address.city": {"$nin": ["Lucknow"]}})
< {
  _id: ObjectId('6ab393571f62337a839beef4'),
  name: 'Riya Verma',
  age: 19,
  course: 'BCA',
  address: {
    city: 'Kanpur',
    state: 'Uttar Pradesh'
  },
  marks: 76
}
{
  _id: ObjectId('6ab393571f62337a839beef5'),
  name: 'Aditya Singh',
  age: 21,
  course: 'BTech',
  address: {
    city: 'Delhi',
    state: 'Delhi'
  },
  marks: 88
}
{
  _id: ObjectId('6ab393571f62337a839beef7'),
  name: 'Rahul Mishra',
  age: 22,
  course: 'BCA',
  address: {
    city: 'Prayagraj',
    state: 'Uttar Pradesh'
  },
  marks: 93
}
```

Fig 12.1 — Students not belonging to Lucknow using \$nin

13. AND OPERATION — \$and

MongoDB Query:

```
db.Students2.find({ $and: [ { course: { $eq: "BSc" } }, { name: "Ishita Agarwal" } ] })
```

Explanation: The \$and operator returns documents that satisfy all the given conditions at the same time. This query finds the student who is enrolled in the BSc course and whose name is Ishita Agarwal — both conditions must be true together.

```
> db.Students2.find({
  $and : [
    { course : { $eq : "BSc" } },
    { name : "Ishita Agarwal" }
  ]
})
< {
  _id: ObjectId('6ab393571f62337a839beefc'),
  name: 'Ishita Agarwal',
  age: 19,
  course: 'BSc',
  address: {
    city: 'Delhi',
    state: 'Delhi'
  },
  marks: 93
}
College2>
```

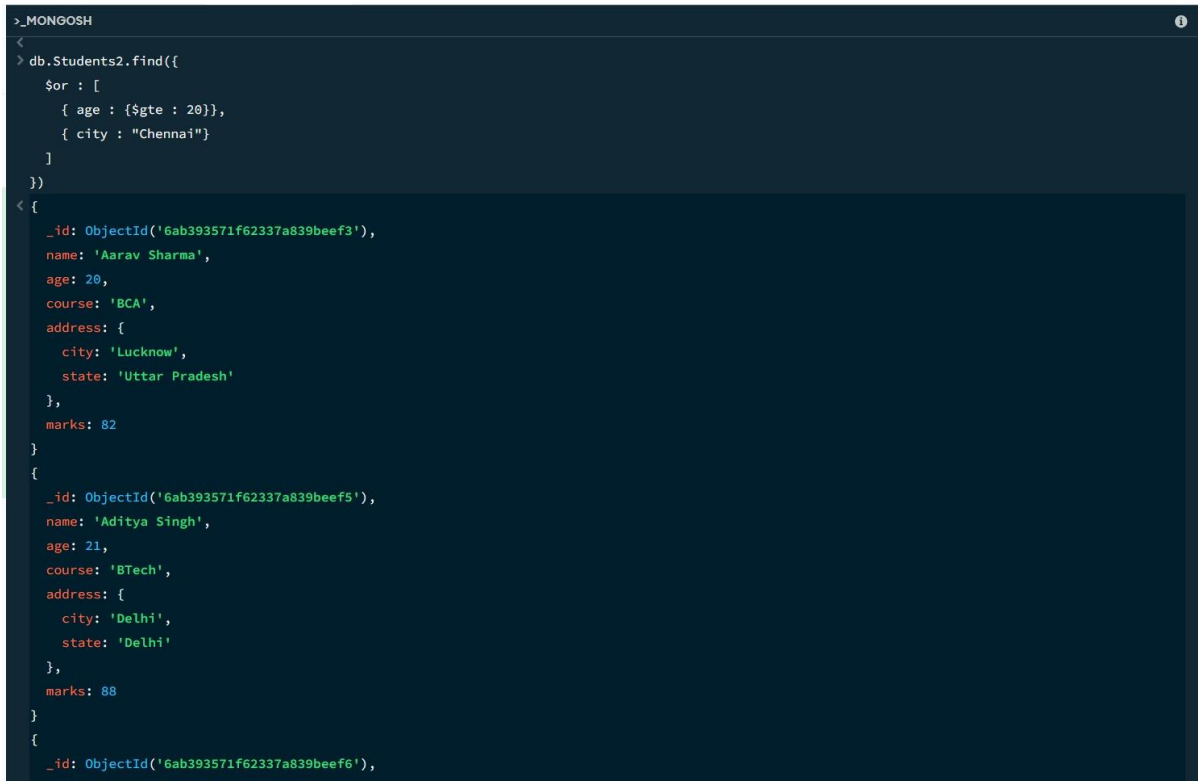
Fig 13.1 — Matching both conditions together using \$and

14. OR OPERATION — \$or

MongoDB Query:

```
db.Students2.find({ $or: [ { age: { $gte: 20 } }, { city: "Chennai" } ] })
```

Explanation: The `$or` operator returns documents that satisfy at least one of the specified conditions. This query displays students who are either 20 years or older, or belong to Chennai, or both.



```
>_MONGOSH
<
> db.Students2.find({
  $or : [
    { age : { $gte : 20 } },
    { city : "Chennai" }
  ]
})
< {
  _id: ObjectId('6ab393571f62337a839beef3'),
  name: 'Aarav Sharma',
  age: 20,
  course: 'BCA',
  address: {
    city: 'Lucknow',
    state: 'Uttar Pradesh'
  },
  marks: 82
}
{
  _id: ObjectId('6ab393571f62337a839beef5'),
  name: 'Aditya Singh',
  age: 21,
  course: 'BTech',
  address: {
    city: 'Delhi',
    state: 'Delhi'
  },
  marks: 88
}
{
  _id: ObjectId('6ab393571f62337a839beef6'),
```

Fig 14.1 — Matching at least one condition using `$or`

15. NOR OPERATION — `$nor`

MongoDB Query:

```
db.Students2.find({ $nor: [ { course: { $eq: "BCA" } }, { "address.state": "Uttar Pradesh" } ] })
```

Explanation: The `$nor` operator returns documents that fail all of the given conditions. This query displays students who are neither enrolled in the BCA course nor belong to the state of Uttar Pradesh.

```
>_MONGOSH
6 | })
> db.Students2.find({
  $nor : [
    { course : {$eq : "BCA"}},
    { "address.state" : "Uttar Pradesh"}
  ]
})
< {
  _id: ObjectId('6ab393571f62337a839bbee5'),
  name: 'Aditya Singh',
  age: 21,
  course: 'BTech',
  address: {
    city: 'Delhi',
    state: 'Delhi'
  },
  marks: 88
}
{
  _id: ObjectId('6ab393571f62337a839bbee6'),
  name: 'Karan Mehta',
  age: 22,
  course: 'BCom',
  address: {
    city: 'Jaipur',
    state: 'Rajasthan'
  },
  marks: 71
}
{
  _id: ObjectId('16ab393571f62337a839bbee7'),
  name: 'Ravi Kumar',
  age: 20,
  course: 'BSc',
  address: {
    city: 'Lucknow',
    state: 'Uttar Pradesh'
  },
  marks: 92
}
```

Fig 15.1 — Records that satisfy none of the given conditions using \$nor

16. SKIP OPERATION

MongoDB Query:

```
db.Students2.find().skip(3)
```

Explanation: The `skip()` method leaves out a given number of documents from the beginning of the result and returns the rest. This query skips the first 3 student records and displays the remaining ones.

```
>_MONGOSH
> db.Students2.find().skip(3)
< {
  _id: ObjectId('6ab393571f62337a839beef6'),
  name: 'Ananya Gupta',
  age: 20,
  course: 'BSc',
  address: {
    city: 'Lucknow',
    state: 'Uttar Pradesh'
  },
  marks: 91
}
{
  _id: ObjectId('6ab393571f62337a839beef7'),
  name: 'Rahul Mishra',
  age: 22,
  course: 'BCA',
  address: {
    city: 'Prayagraj',
    state: 'Uttar Pradesh'
  },
  marks: 69
}
{
  _id: ObjectId('6ab393571f62337a839beef8'),
  name: 'Priya Yadav',
  age: 19,
  course: 'BBA',
  address: {
    city: 'Agra',
    state: 'Uttar Pradesh'
  },
  marks: 85
}
```

Fig 16.1 — Skipping the first 3 records using skip(3)

17. LIMIT OPERATION

MongoDB Query:

```
db.Students2.find().limit(5)
```

Explanation: The `limit()` method restricts the number of documents returned by a query. This command displays only the first 5 student records from the collection.

```
>_MONGOSH
> db.Students2.find().limit(5)
< {
  _id: ObjectId('6ab393571f62337a839beef3'),
  name: 'Aarav Sharma',
  age: 20,
  course: 'BCA',
  address: {
    city: 'Lucknow',
    state: 'Uttar Pradesh'
  },
  marks: 82
}
{
  _id: ObjectId('6ab393571f62337a839beef4'),
  name: 'Riya Verma',
  age: 19,
  course: 'BCA',
  address: {
    city: 'Kanpur',
    state: 'Uttar Pradesh'
  },
  marks: 76
}
{
  _id: ObjectId('6ab393571f62337a839beef5'),
  name: 'Aditya Singh',
  age: 21,
  course: 'BTech',
  address: {
    city: 'Delhi',
    state: 'Delhi'
  },
  marks: 85
}
```

Fig 17.1 — Displaying only 5 records using limit(5)

18. COUNT DOCUMENTS

MongoDB Query:

```
db.Students2.find().count()
```

Explanation: The count() method returns the total number of documents that match a given query. Here, it returns the total number of student records currently present in the Students2 collection, which is 52.

```
> db.Students2.find().count()
< 52
college2>
```

Fig 18.1 — Total number of documents in the collection

19. UPDATE OPERATION — updateOne()

MongoDB Query:

```
db.Students2.updateOne( { _id: ObjectId("6ab393571f62337a839bef1c") }, {
  $set: { marks: 72 } } )
```

Explanation: The updateOne() method updates the first document that matches the given filter. Here, the marks field of the student having the specified _id is changed to 72 using the \$set operator, which modifies only the specified field without affecting the rest of the document.

```

> db.Students2.updateOne({_id: ObjectId('6ab393571f62337a839bef1c')}, {$set: {marks: 72}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

Fig 19.1 — Updating a student's marks using updateOne()

20. VERIFYING THE UPDATE

MongoDB Query:

```
db.Students2.find({ _id: ObjectId("6ab393571f62337a839bef1c") })
```

Explanation: To confirm that the update was successful, the same document is fetched again using its `_id`. The output shows that the student's marks field has indeed been updated to 72.

```

> db.Students2.find({_id: ObjectId('6ab393571f62337a839bef1c')})
< {
  _id: ObjectId('6ab393571f62337a839bef1c'),
  name: 'Anjali Mehta',
  age: 19,
  course: 'BBA',
  address: {
    city: 'Ahmedabad',
    state: 'Gujarat'
  },
  marks: 72
}
College2>

```

Fig 20.1 — Verifying the updated record

21. DELETE OPERATION — deleteOne()

MongoDB Query:

```
db.Students2.deleteOne({ _id: ObjectId("6ab393571f62337a839bef1c") })
```

Explanation: The `deleteOne()` method removes the first document that matches the given filter. Here, the student document with the specified `_id` is permanently removed from the collection. The output field `deletedCount: 1` confirms that exactly one document was deleted.

```

> db.Students2.deleteOne({_id: ObjectId('6ab393571f62337a839bef1c')})
< {
  acknowledged: true,
  deletedCount: 1
}
College2>

```

Fig 21.1 — Deleting a student record using deleteOne()

MONGODB OPERATORS AND METHODS USED

Operator	Meaning	Example
\$eq	Equal to	age: {\$eq: 19}
\$gt	Greater than	marks: {\$gt: 80}
\$lt	Less than	marks: {\$lt: 80}
\$gte	Greater than or equal to	age: {\$gte: 22}
\$lte	Less than or equal to	marks: {\$lte: 70}
\$ne	Not equal to	city: {\$ne: 'Meerat'}
\$in	Matches any value in array	address.city: {\$in: [...] }
\$nin	Matches none of the values in array	address.city: {\$nin: [...] }
\$and	All conditions must be true	\$and: [...]
\$or	At least one condition true	\$or: [...]
\$nor	None of the conditions true	\$nor: [...]
skip()	Skips a given number of documents	find().skip(3)
limit()	Restricts number of documents returned	find().limit(5)
count()	Counts total matching documents	find().count()

CONCLUSION

This project provided practical, hands-on knowledge of NoSQL databases using MongoDB. A College Student Record Management System containing 52 student documents was created inside the College2 database, and a variety of MongoDB operations were performed on this data using MongoDB Compass and the mongosh shell.

Through this project, I learned how to insert multiple documents at once, retrieve and format records, update and delete specific documents, and filter data using comparison operators such as \$eq, \$gt, \$lt, \$gte, \$lte, \$ne, \$in and \$nin. I also learned how the logical operators \$and, \$or and \$nor combine multiple conditions, and how skip(), limit() and count() help control and summarise query results.

Overall, this project helped me understand how MongoDB can be used to store, manage and query real-world, document-based data efficiently, and strengthened my foundation in NoSQL database concepts.